

MLA03 – Assessment Tool 2: Reinforcement Learning Case Study Solutions (CO1)

1) A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays.

(a) Markov Decision Process (MDP) Components

1. States (S)

The state represents the current condition of the warehouse robot.

Examples:

- Robot's current position
- Item location
- Destination location
- Battery level
- Obstacle positions
- Current load (empty/full)
- Distance to destination
- Traffic in warehouse

Example State:

S = (Position=A5, Battery=80%, Carrying=Box, Obstacle=Nearby)

2. Actions (A)

The actions are all possible operations the robot can perform.

Examples:

- Move Forward
- Move Backward
- Turn Left
- Turn Right
- Pick Item
- Place Item
- Charge Battery
- Wait

3. Environment (E)

The environment includes everything with which the robot interacts.

Components:

- Warehouse layout
- Shelves
- Packages
- Human workers
- Other robots
- Charging stations
- Obstacles

4. Reward (R)

The reward guides the robot toward desirable behavior.

Positive Rewards

- Correctly picks item → +15
- Delivers package successfully → +30
- Reaches destination quickly → +10
- Avoids obstacle → +5

- Chooses shortest path → +8

Negative Rewards

- Collision → -50
- Wrong package → -25
- Delay → -10
- Battery drained → -30
- Unnecessary movement → -2

5. Policy (π)

A policy defines the best action in every state.

Example:

IF	obstacle	detected
→ Turn Left		
IF	item	reached
→ Pick Item		
IF	battery	<20%
→ Move to charging station		

(b) Reward Function

A balanced reward function is

Reward = Speed + Accuracy + Safety – Penalties

Mathematically,

$$R = 10(\text{Speed}) + 20(\text{Accuracy}) + 15(\text{Safety}) - 50(\text{Collision}) - 10(\text{Delay})$$

This reward encourages

- Fast delivery
- Correct package handling
- Safe navigation
- Energy efficiency

(c) Influence of Reward Design

A good reward function helps the robot:

- Learn optimal routes
- Avoid collisions
- Reduce delivery time
- Improve picking accuracy
- Save battery power

Poor reward design may cause:

- Robot moving too fast and crashing
- Ignoring safety
- Taking unnecessary shortcuts
- Focusing only on one objective

Hence, balanced rewards improve overall warehouse efficiency.

Conclusion

Using an MDP allows the warehouse robot to learn optimal actions through interaction with the environment. A carefully designed reward function balances speed, safety, and accuracy, resulting in efficient warehouse operations.

CASE STUDY 2

A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history.

(a) State Representation

The state contains user information.

Examples:

- Age
- Preferred genres
- Recently watched movies
- Watch time
- Ratings
- Search history
- Device type
- Time of day

Example:

State

=

(User likes Action, watched Avengers yesterday, evening)

(b) Action Space

Possible actions are recommending different movies.

Examples:

- Recommend Action movie
- Recommend Comedy
- Recommend Drama
- Recommend Horror
- Recommend Trending movie
- Recommend Similar movie

(c) Reward Function

Positive Rewards

User	watches	recommended	movie
→ +20			

User	watches	full	movie
→ +30			

User		gives	Like
→ +10			

Subscription			renewed
→ +40			

Negative Rewards

Movie			skipped
→ -15			

Movie			disliked
→ -20			

App	closed		immediately
→ -25			

(d) Effect of Different Reward Signals

Positive rewards

- Increase user satisfaction
- Improve watch time
- Increase retention

Negative rewards

- Prevent poor recommendations
- Reduce irrelevant suggestions

Long-term rewards

- Increase subscriptions
- Improve customer loyalty

(e) Exploration vs Exploitation

Exploration

The system recommends new or less-known movies.

Advantages:

- Discovers new interests
- Avoids repetitive recommendations

Disadvantages:

- User may dislike suggestions

Exploitation

The system recommends movies similar to those already liked.

Advantages:

- Higher immediate satisfaction
- Better click rate

Disadvantages:

- Less diversity
- Recommendation becomes repetitive

Best Strategy

Use ϵ -Greedy Exploration.

- 90% recommend favorite genres
- 10% recommend new genres

This balances user satisfaction with discovering new preferences.

Conclusion

RL enables personalized movie recommendations by continuously learning user preferences. A balanced reward function and an appropriate exploration–exploitation strategy improve engagement, watch time, and customer retention.

CASE STUDY 3

An autonomous agricultural system uses Reinforcement Learning (RL) to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions, and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm.

(a) RL Framework

1. States (S)

A state represents the current condition of the agricultural field.

Examples of state variables:

- Soil moisture level
- Soil nutrient level (Nitrogen, Phosphorus, Potassium)
- Temperature
- Humidity
- Weather forecast (Rain/No Rain)
- Crop growth stage
- Water availability
- Previous irrigation amount

Example State:

S = (Soil Moisture = 35%, Temperature = 30°C, Rain Expected = Yes, Crop Stage = Flowering)

2. Actions (A)

The RL agent selects actions to improve crop growth.

Possible actions:

- Increase irrigation
- Decrease irrigation
- Stop irrigation
- Apply fertilizer
- Reduce fertilizer amount
- Delay fertilizer application
- No action

3. Environment (E)

The environment consists of all external factors affecting farming.

Components:

- Agricultural field
- Soil conditions
- Weather
- Water resources
- Crop health
- Fertilizer availability
- Climate changes

4. Reward Function (R)

The reward should encourage high crop yield while minimizing water and fertilizer usage.

Positive Rewards

- Healthy crop growth → +20
- Increased crop yield → +40

- Water saved → +15
- Proper fertilizer usage → +10
- Efficient irrigation scheduling → +15

Negative Rewards

- Over-irrigation → -20
- Under-irrigation → -25
- Fertilizer wastage → -15
- Crop damage → -40
- Water shortage → -20

5. Policy (π)

The policy decides the best farming action for each state.

Example:

- If soil moisture < 30% → Increase irrigation.
- If rain is expected → Delay irrigation.
- If nutrient level is low → Apply fertilizer.
- If soil moisture is optimal → No action.

(b) Effect of Delayed Rewards

In agriculture, rewards are often delayed because crop growth takes weeks or months.

Example

Today:

- The system irrigates the field.

After several weeks:

- Crop growth improves.
- Final crop yield increases.

The reward is received much later than the action.

Effects of Delayed Rewards

Advantages

- Encourages long-term planning.
- Produces higher crop yields.
- Reduces unnecessary resource usage.
- Improves sustainability.

Challenges

- Difficult to identify which action caused the final result.
- Slower learning process.
- Requires more training episodes.
- Weather changes can affect outcomes.

Therefore, RL must learn to associate early actions with future rewards.

(c) Suitable Learning Algorithm

The most suitable algorithm is **Q-Learning**.

Reasons

- Learns the optimal policy through trial and error.
- Does not require a predefined model of the environment (model-free).
- Works well with delayed rewards using the Bellman Equation.
- Can optimize long-term cumulative rewards.
- Suitable for dynamic agricultural environments.

For very large or continuous state spaces, **Deep Q-Network (DQN)** can be used because it combines Q-Learning with deep neural networks.

(d) Analysis

The RL agent continuously observes soil and weather conditions, chooses irrigation and fertilizer actions, and receives rewards based on crop performance. Since crop yield is observed only after a long period, delayed rewards make learning challenging. Q-Learning helps the agent learn the best long-term strategy by maximizing cumulative rewards, leading to efficient resource usage, higher productivity, and sustainable farming.

Conclusion

Reinforcement Learning provides an intelligent solution for precision agriculture by optimizing irrigation and fertilizer usage. A well-designed state space, action set, and reward function enable efficient decision-making. Although delayed rewards slow learning, algorithms like Q-Learning effectively maximize long-term crop yield while conserving water and fertilizer resources.

CASE STUDY 4 (5 Marks)

A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making.

(a) Modeling States and Environment

1. States (S)

The state represents the current status of the computer network.

Examples:

- Network traffic volume
- Number of failed login attempts
- IP address reputation
- Firewall alerts
- CPU usage
- Malware detection status
- Suspicious packet count
- User authentication status

Example State:

S = (Traffic = High, Failed Logins = 20, Firewall Alert = ON, Malware = Detected)

2. Environment (E)

The environment includes all network components and external conditions.

Components:

- Computer network
- Servers
- Routers
- Firewalls
- Connected devices
- Users
- Internet traffic
- Cyber attackers

The RL agent interacts continuously with this environment to detect and respond to attacks.

(b) Actions (A)

Possible actions taken by the RL agent:

- Allow network traffic
- Block suspicious IP address
- Quarantine infected device
- Generate security alert
- Request user authentication
- Increase firewall security
- Start malware scan
- Monitor activity

(c) Reward Function (R)

Positive Rewards

- Correctly detect attack → +30
- Block malicious traffic → +40
- Prevent data breach → +50
- Identify malware → +25
- Maintain normal service → +15

Negative Rewards

- Missed attack → -50
- False alarm → -15
- Blocking legitimate users → -25
- Slow response → -20
- System downtime → -40

(d) Sparse Rewards

Sparse rewards occur because attacks are relatively rare.

Challenges

- The agent receives rewards infrequently.
- Learning becomes slow.
- Difficult to determine which actions were effective.
- More training time is required.

Solutions

- Reward shaping.
- Simulated cyber attacks for training.
- Experience replay.
- Deep Reinforcement Learning.

(e) Real-Time Decision Making

The cybersecurity agent must respond immediately to threats.

Challenges

- High-speed network traffic.
- Large number of users.
- Continuous attacks.
- Limited processing time.
- Balancing security with system performance.

Importance

Fast decisions help:

- Prevent data theft.
- Minimize damage.
- Maintain service availability.
- Protect sensitive information.

(f) Analysis

The RL agent continuously monitors network activity, selects defensive actions, and learns from rewards. A balanced reward function encourages accurate intrusion detection while minimizing false alarms. Sparse rewards make learning difficult because attacks occur infrequently, and real-time requirements demand quick, accurate decisions. Deep RL techniques can improve performance in such dynamic environments.

Conclusion

Reinforcement Learning enhances cybersecurity by enabling adaptive intrusion detection and automated response. Properly designed states, actions, and rewards allow the agent to learn effective defense strategies. Addressing sparse rewards and real-time constraints is essential for building a reliable and efficient cybersecurity system.

CASE STUDY 5

A ride-sharing company plans to use Reinforcement Learning (RL) to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies.

(a) RL Framework

1. States (S)

The state represents the current condition of the ride-sharing system.

Examples of state variables:

- Number of available drivers
- Number of ride requests
- Passenger locations
- Driver locations
- Traffic conditions
- Weather conditions
- Time of day
- Current fare price
- Peak or off-peak hours

Example State:

S = (Drivers = 50, Ride Requests = 80, Traffic = Heavy, Weather = Rainy, Time = 6 PM)

2. Actions (A)

The RL agent selects actions to optimize service.

Possible actions:

- Assign the nearest driver
- Reassign another driver
- Increase ride fare (surge pricing)
- Decrease fare
- Offer discounts
- Move idle drivers to high-demand areas

- Accept or reject ride requests

3. Environment (E)

The environment includes all external factors affecting ride-sharing.

Components:

- Drivers
- Passengers
- Road network
- Traffic conditions
- Weather
- GPS system
- Payment system
- City events

4. Reward Function (R)

The reward should maximize customer satisfaction and company profit while minimizing waiting time.

Positive Rewards

- Successful ride completed → +30
- Reduced passenger waiting time → +20
- Efficient driver allocation → +15
- High customer rating → +20
- Increased company profit → +15

Negative Rewards

- Ride cancellation → -25
- Long waiting time → -20
- Driver idle time → -15
- Customer complaint → -30
- Unfair surge pricing → -20

5. Policy (π)

The policy determines the best action for each situation.

Example:

- If demand is high and drivers are few → Increase fare slightly and move nearby drivers.
- If demand is low → Reduce fare to attract passengers.
- If traffic is heavy → Assign the closest available driver.

(b) Effect of Environmental Uncertainty

Ride-sharing environments are highly dynamic and uncertain.

Sources of Uncertainty

- Sudden traffic congestion
- Bad weather
- Festivals and public events
- Driver cancellations
- Unexpected increase in ride requests
- GPS inaccuracies

Impact on Learning

- State changes rapidly.
- Optimal actions change frequently.
- Learning becomes more difficult.

- Requires continuous updates and adaptation.
- More exploration is needed to handle changing conditions.

RL agents must continuously learn from new experiences to perform well in uncertain environments.

(c) Ethical Concerns in Dynamic Pricing

Dynamic pricing (surge pricing) increases fares during periods of high demand.

Ethical Issues

1. **Unfair Pricing**
 - Extremely high fares during emergencies or disasters can exploit customers.
2. **Customer Dissatisfaction**
 - Passengers may feel they are being overcharged.
3. **Income Inequality**
 - Low-income users may not afford transportation during peak hours.
4. **Lack of Transparency**
 - Customers may not understand why prices suddenly increase.
5. **Discrimination**
 - Pricing should not unfairly target specific locations or user groups.

Possible Solutions

- Set a maximum limit on surge pricing.
- Clearly explain fare increases.
- Offer discounts during emergencies.
- Ensure fair and transparent pricing policies.
- Regularly audit the RL system for bias.

(d) Analysis

The RL agent learns the best driver allocation and pricing strategies by interacting with the environment. Environmental uncertainty requires continuous learning and adaptation. A balanced reward function improves efficiency, reduces waiting time, and increases customer satisfaction. Ethical safeguards are necessary to ensure fairness and maintain public trust.

Conclusion

Reinforcement Learning can significantly improve ride-sharing services through intelligent driver allocation and dynamic pricing. However, the system must carefully balance profitability with customer fairness, transparency, and ethical responsibility.